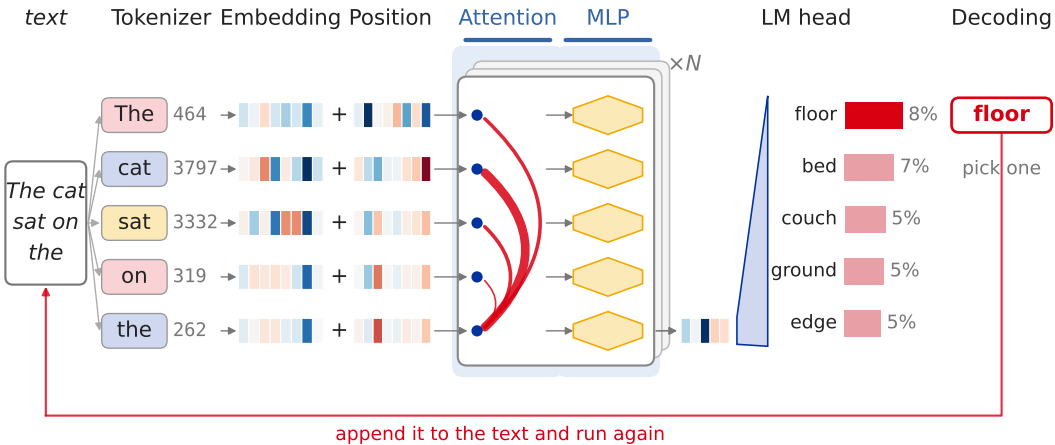
Anatomy of a 2026 LLM

What changed since GPT-2, and what each change saves

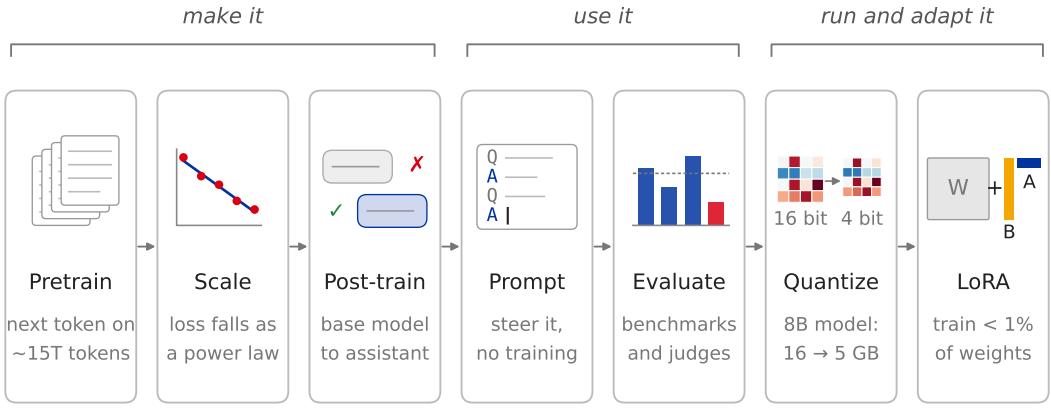
LLM chapter, session 14. Model facts from published configuration files, read 2026-10-06.

Where we are: one forward pass



Every box is still there. Today: what 2026 models changed inside them.

Where we are: the life of a model



The last lecture of the chapter: the whole life, seen through one modern model.

Two configuration files

	GPT-2 small (2019)	**gpt-oss-20b (2025)**
layers	12	24
attention heads (size)	12 (64)	64 (64)
key/value heads	12	**8**
attention span	every layer: all tokens	**every other layer: last 128**
feed-forward	one MLP	**32 experts, 4 used per token**
parameters	124M	21B stored, **3.6B used per token**
context	1,024	131,072

Predict first: the bold entries are the new ideas (each one is explained today; for now, guess from the words). What do they have in common?

They all save memory or compute when the model runs

Answer: each one makes the model cheaper to run.

- **8 key/value heads, not 64**: groups of 8 query heads share one key and one value, so the KV cache (LLM-12) is 8× smaller. Section "The KV cache".
- **A 128-token window in half the layers**: those layers look only at the last 128 tokens, so their cache stops growing. Same section.
- **Experts**: many feed-forward parts, of which each token uses 4: 21B parameters of knowledge, 3.6B of work per token. Section "Compute and traffic".

The block of LLM-4 is still recognisable: residual stream, attention, a feed-forward part, normalisation. What changed is mostly *how much each part costs to run*.

Training happens once; a model answers billions of times. So almost every change since 2019 is a way to pay less per token. Today: each one with its bill, from a real config.

Outline

The KV cache

Compute and traffic

Serving many users

What did not change

The KV cache

Long contexts are expensive. Here is why, and the fixes.

Prefill and decode

- **Prefill**: the whole prompt goes through in one pass, all positions in parallel (LLM-4). Big matrix multiplications: limited by arithmetic.
- **Decode**: one new token per pass. Each pass reads all the weights *and* the KV cache of every earlier token: limited by memory bandwidth (LLM-12).
- The KV cache grows with every token (LLM-12). Bytes per token: $2_{\text{K,V}} \times$ layers $\times$ KV heads $\times$ head size $\times\, 2_{\text{bytes}}$ - one 2 for keys and values, one for 16-bit numbers.

Every byte of cache per token is read again for every new token. Shrinking it makes long conversations both fit and run faster.

Fewer key/value heads

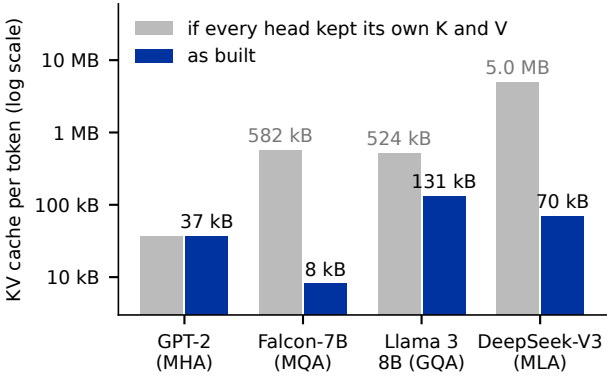
- **Multi-head** (LLM-3): every query head has its own keys and values.
- **Multi-query** (MQA): all query heads share *one* key/value head.
- **Grouped-query** (GQA): query heads share in groups - Llama 3 8B: 32 query heads in 8 groups of 4, each group sharing one key head and one value head.

By hand, bytes per token (16-bit):

	as built	**if multi-head**
Falcon-7B (MQA: 1 of 71)	$2 \times 32 \times 1 \times 64 \times 2 = 8{,}192$	581,632 (71×)
Llama 3 8B (GQA: 8 of 32)	$2 \times 32 \times 8 \times 128 \times 2 = 131{,}072$	524,288 (4×)

One shared head saves the most and costs some quality; GQA is the compromise most current models chose (gpt-oss: 8).

Cache a compressed version: MLA



Multi-head latent attention (DeepSeek): per layer and token, cache one *latent* vector - a compressed summary the model learns to make. Every head's keys and values are rebuilt from it when needed.

The bill: extra matrix multiplications for the rebuild. Decode waits on memory, not arithmetic (LLM-12), so it is a good trade.

MLA by hand: DeepSeek-V3

- **As built**, per layer: a 512-number latent plus a 64-number key part that carries the position (RoPE, LLM-2). $576 \times 2_{\text{bytes}} \times 61$ layers $= 70{,}272$ bytes per token. No $2_{K,V}$: keys and values both come from the one vector.
- **If multi-head**: 128 heads, each with a key of $128 + 64 = 192$ numbers and a value of 128. $128 \times (192 + 128) \times 2_{\text{bytes}} \times 61 = 4{,}997{,}120$ bytes, about 5.0 MB per token.
- Ratio: 71×. Falcon's MQA also saved 71×, by coincidence: 71 heads there, a 576-number vector against 128 heads of 320 numbers here.

At 128K tokens: 9.2 GB of cache instead of 655 GB. Without MLA, one long conversation's cache would be about as big as the whole model's 671B weights in 8-bit.

Config: DeepSeek-V3 (kv_lora_rank 512, qk_rope_head_dim 64, qk_nope_head_dim 128, v_head_dim 128, 61 layers).

Look back less: sliding windows

- A **sliding-window** layer attends only to the last w tokens, so its cache stops growing at w.
- gpt-oss-20b alternates: 12 layers with a 128-token window, 12 layers with full attention. The full layers keep the long-range view.
- **By hand**, per layer and token: $2_{\text{K,V}} \times 8$ KV heads $\times\, 64 \times 2_{\text{bytes}} = 2{,}048$ bytes. At 128K tokens, all 24 layers full: $24 \times 2{,}048 \times 131{,}072 = 6.4$ GB. Alternating: the 12 windowed layers hold only 128 tokens each, so about 3.2 GB.

Half the cache, but not for free: a windowed layer loses direct access to old tokens. The full layers in between carry the long-range information. That trade is a design choice, tested on benchmarks.

Or a fixed-size memory: linear attention

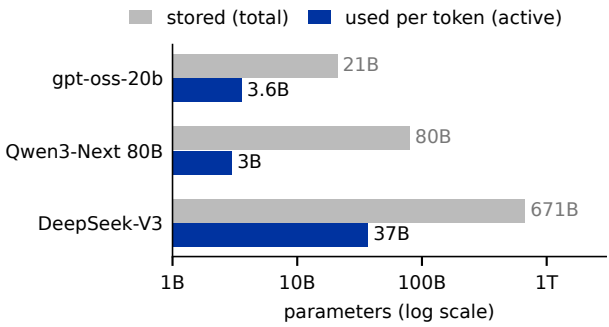
- Some layers replace attention with a **linear-attention** layer that keeps a fixed-size state, updated token by token - an RNN's memory (L21's "recurrence comeback").
- Qwen3-Next-80B-A3B: 48 layers, full attention in every 4th layer only (12 of 48). Only those keep a growing cache:
  $2_{\text{K,V}} \times 12$ layers $\times$ 2 KV heads $\times$ 256 $\times$ $2_{\text{bytes}} = 24{,}576$ bytes per token. The linear layers' fixed-size state comes on top, but it does not grow with the context.
- The cost: a fixed-size state forgets; the full-attention layers in between recall exactly.

Four answers to one bill: share heads (GQA), compress (MLA), look back less (windows), remember in fixed size (linear attention). Real 2026 models mix them.

Compute and traffic

Big knowledge, small work per token, fewer trips to memory.

Mixture of experts



Replace the one feed-forward MLP with many smaller ones, the **experts**. A small **router** scores them for each token and sends the token to the top few.

gpt-oss-20b: 32 experts, 4 per token.
Qwen3-Next: 512, 10 per token.
DeepSeek-V3: 256 routed, 8 per token, plus 1 **shared** expert that every token uses.

DeepSeek-V3 stores 671B parameters and uses 37B per token: it stores 18× more than it uses. MoE buys a big model's knowledge at a small model's cost per token, and pays in memory.

MoE by hand: what is saved, and when

- **Arithmetic**: a forward pass costs about 2 FLOPs per *active* parameter (LLM-8). gpt-oss-20b: about 7 GFLOPs per token, against 42 if all 21B were used.
- **Memory traffic**, with one user: each token reads only the weights it uses (3.6B of 21B), so decode, which waits on memory, speeds up too.
- **In a big batch** ("Serving many users"): different tokens pick different experts, so most experts get read anyway. The saving is then in arithmetic only.

Memory must hold all 21B either way (or 4-bit, LLM-12). MoE saves work per token, not space.

Keeping the experts busy

- A router left alone sends most tokens to a few favourite experts; the rest never learn, and the favourites become a bottleneck.
- The usual fix is an extra loss that rewards balanced use. DeepSeek-V3 instead adjusts a per-expert bias during training ("auxiliary-loss-free").
- **Shared experts** (DeepSeek-V3 has one): they learn what all tokens need, so the routed experts can specialise.

Experts are not "the maths expert" and "the history expert". They specialise in patterns the router finds useful, which rarely match human topics.

FlashAttention: the same numbers, fewer trips

- The attention bill was quadratic (LLM-5). In training and prefill the full $n \times n$ score matrix exists, and on a GPU the slow part is often moving it to and from memory, not computing it. (In decode the scores are a single row.)
- A GPU has a little fast memory on the chip (SRAM) and a lot of slower main memory (HBM). On an A100, SRAM has about $10\times$ HBM's bandwidth and is about $2000\times$ smaller.
- FlashAttention (Dao et al., 2022) works through small blocks (tiles) of queries and keys that fit in the fast memory, and never writes the full $n \times n$ matrix to main memory.

Exact, not an approximation: the same outputs and about the same arithmetic, much less memory traffic. A big part of why long contexts became practical to train.

More than one token per step

- Decoding writes one token per pass. LLM-6's **speculative decoding**: a small draft model guesses several tokens, the big model checks them in one pass, and the output stays exactly the big model's.
- **Multi-token prediction**: DeepSeek-V3 adds one extra Transformer block that takes the main model's hidden state and predicts the token *after* next. It shares the main model's embedding and output head. It helps training, and at inference it serves as the draft.
- Measured by DeepSeek: the drafted second token is accepted 85-90% of the time, for 1.8× the tokens per second.

Both turn the bandwidth-bound decode into fewer, fuller passes.

Serving many users

The same tricks, at scale.

Batching: shared weights, private caches

- One user leaves the GPU mostly idle (decode is bandwidth-bound). Batching many users reads the weights once per step for all of them. But each user has their own KV cache.
- **By hand**, Llama 3 8B: the weights, $8\text{B} \times 2_{\text{bytes}} = 16$ GB, shared. One user at 4,096 tokens: $131{,}072 \times 4{,}096 \approx 0.54$ GB of cache. About 30 such users need as much memory for caches as for the weights.
- **Continuous batching**: requests join and leave the batch at every step, instead of waiting for the slowest one to finish.

With many users, the caches, not the weights, fill the GPU. So serving is mostly about cache memory.

PagedAttention: cache memory in pages

- Without it, each request reserves one big block of cache memory, sized for the longest answer it might give. Much of that block stays empty.
- **PagedAttention** (Kwon et al., 2023, vLLM): store each request's cache in small fixed-size pages, the way an operating system hands out memory. A request that needs 3 pages gets any 3 free pages, and more as it grows.
- Near-zero wasted cache memory, so more users fit in a batch: 2-4$\times$ the throughput of earlier serving systems (FasterTransformer, Orca) at the same latency.

These are the server engines of LLM-12. On a laptop, with one user, they matter less.

What did not change

The 2017 core.

What did not change

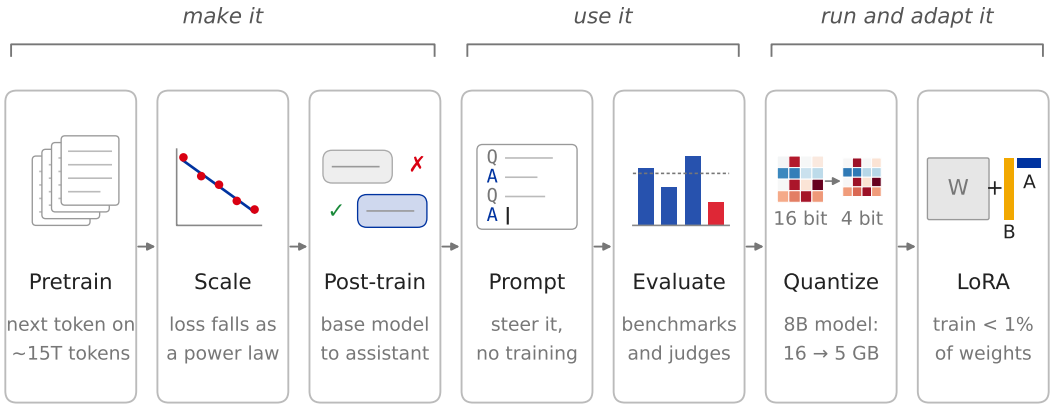
- The residual stream with attention and feed-forward parts added to it (LLM-4).
- Next-token pretraining on filtered web text (LLM-7), then post-training (LLM-9).
- Tokens, embeddings, a softmax over the vocabulary at the end (LLM-1, LLM-2, LLM-5).

A student of 2017's Transformer can read a 2026 configuration file. Most of the new words in it - GQA, MLA, sliding window, experts - name a way to make the same computation cheaper to run.

Recap

- **Inference cost drives the design**: decode is bandwidth-bound, and the KV cache grows with every token.
- **Smaller cache**: GQA (Llama 3 8B, 4×), MQA (Falcon-7B, 71×), MLA (DeepSeek-V3, also 71×), sliding windows (gpt-oss: half), linear-attention layers (Qwen3-Next: 1 in 4 full).
- **Less work and traffic**: MoE (gpt-oss-20b: 3.6B of 21B per token), FlashAttention (fewer memory trips), multi-token prediction (1.8× tokens per second) and speculative decoding.
- **Serving**: shared weights, private caches - continuous batching and paged KV memory.
- **Unchanged**: the residual stream, attention, next-token training.

The end of the map



The whole chapter: tokens to a 2026 model.

Sources & further reading

- Configuration files (Hugging Face, read 2026-10-06): openai/gpt-oss-20b, deepseek-ai/DeepSeek-V3, tiiuae/falcon-7b, Qwen/Qwen3-Next-80B-A3B-Instruct; Llama 3 paper, Table 3.
- Shazeer (2019), multi-query attention; Ainslie et al. (2023), GQA; DeepSeek-AI (2024), DeepSeek-V2 and V3 (MLA, MoE, multi-token prediction).
- Dao et al. (2022), "FlashAttention"; Kwon et al. (2023), "Efficient Memory Management for Large Language Model Serving with PagedAttention".

Figures: `py_src/anatomy_figs.py`. Every number computed by hand from the configs above and asserted there.